provide answer Exercises

1. Implement a generic class called Box that can hold any type of object.

a) Create a class Box with a type parameter T.

b) Add methods to set and get the value of the box.

c) Write a test class to create instances of Box with different types (e.g., Integer, String).

2. Write a program that uses ArrayList and HashMap.

a) Create an ArrayList of String and add some elements.

b) Iterate over the ArrayList and print each element.

c) Create a HashMap to store names as keys and ages as values.

d) Add some entries to the HashMap and print each key-value pair.

3. Write a program that sorts a list of custom objects using a Comparator.

a) Create a class Person with name and age fields.

b) Implement a Comparator to sort the Person objects by age.

c) Use Collections.sort to sort the list and print the sorted list.

4. Create a generic collection class GenericCollection that can hold any type of Collection.

a) Define methods to add an element, remove an element, and print all elements.

b) Test the class with different types of collections (ArrayList, HashSet).

5. Create a generic class StudentList that holds a collection the accepts only objects of Student class.

6. Write a function to_uppercase(s) that returns the uppercase version of the string s.

Test cases:

a) Test converting a normal string to uppercase.

b) Test converting an empty string.

c) Test converting a string with mixed case.

d) Test converting a string with numbers and special characters.

e) Test converting a string with spaces.

7. Write a function remove_spaces(s) that returns the string s with all spaces removed.

Test cases:

a) Test removing spaces from a normal string.

b) Test removing spaces from an empty string.

c) Test removing spaces from a string with no spaces.

d) Test removing spaces from a string with leading and trailing spaces.

e) Test removing spaces from a string with special characters and numbers.

8. Write code snippets demonstrating different ways to iterate over elements in an ArrayList, LinkedList, and HashMap in Java. Utilize for loops, enhanced for loops, and iterators.

9. Implement a method in Java that takes two sorted lists of integers and merges them into a single sorted list. Utilize two iterators and a temporary list if needed.

10. Write methods in Java that utilize wildcards to achieve lower and upper bounded restrictions on generic types. Demonstrate the use of <? extends T> and <? super T> in method signatures.

1. Generic Box Class

java

CopyEdit

```
// a & b: Generic Box class
class Box<T> {
   private T value;

   public void set(T value) {
      this.value = value;
   }
```

```
    public T get() {
        return value;
    }
}

// c: Test class
public class TestBox {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        System.out.println("Integer value: " + intBox.get());

        Box<String> strBox = new Box<>();
        strBox.set("Hello");
        System.out.println("String value: " + strBox.get());
    }
}
```

---

2. ArrayList and HashMap Example

java

CopyEdit

```
import java.util.*;

public class ListMapExample {
    public static void main(String[] args) {
        // a & b: ArrayList
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
```

```
        list.add("Orange");

        for (String item : list) {
            System.out.println("Fruit: " + item);
        }

        // c & d: HashMap
        HashMap<String, Integer> map = new HashMap<>();
        map.put("Alice", 30);
        map.put("Bob", 25);
        map.put("Charlie", 35);

        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " is " + entry.getValue() + " years old");
        }
    }
}
```

---

3. Sorting with Comparator

java

CopyEdit

```
import java.util.*;

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
```

```
        this.age = age;
    }

    public String toString() {
        return name + " (" + age + ")";
    }
}

class AgeComparator implements Comparator<Person> {
    public int compare(Person a, Person b) {
        return Integer.compare(a.age, b.age);
    }
}

public class SortPerson {
    public static void main(String[] args) {
        List<Person> people = new ArrayList<>();
        people.add(new Person("Alice", 30));
        people.add(new Person("Bob", 25));
        people.add(new Person("Charlie", 35));

        Collections.sort(people, new AgeComparator());
        for (Person p : people) {
            System.out.println(p);
        }
    }
}
```

---

4. GenericCollection Class

java

CopyEdit

```
import java.util.*;

class GenericCollection<T> {
    private Collection<T> collection;

    public GenericCollection(Collection<T> collection) {
        this.collection = collection;
    }

    public void add(T item) {
        collection.add(item);
    }

    public void remove(T item) {
        collection.remove(item);
    }

    public void printAll() {
        for (T item : collection) {
            System.out.println(item);
        }
    }
}

class TestGenericCollection {
    public static void main(String[] args) {
        GenericCollection<String> list = new GenericCollection<>(new ArrayList<>());
```

```
        list.add("Apple");
        list.add("Banana");
        list.printAll();

        GenericCollection<Integer> set = new GenericCollection<>(new HashSet<>());
        set.add(10);
        set.add(20);
        set.printAll();
    }
}
```

---

5. StudentList Generic Class

java

CopyEdit

```
import java.util.*;

class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    public String toString() {
        return name;
    }
}

class StudentList {
```

```
    private List<Student> students = new ArrayList<>();

    public void add(Student s) {
        students.add(s);
    }

    public void printAll() {
        for (Student s : students) {
            System.out.println(s);
        }
    }
}
```

---

6. to_uppercase Function

java

CopyEdit

```
public class StringUtils {
    public static String to_uppercase(String s) {
        return s.toUpperCase();
    }

    public static void main(String[] args) {
        System.out.println(to_uppercase("hello"));          // HELLO
        System.out.println(to_uppercase(""));               // (empty)
        System.out.println(to_uppercase("HeLLo"));          // HELLO
        System.out.println(to_uppercase("abc123!@#"));      // ABC123!@#
        System.out.println(to_uppercase("hello world"));    // HELLO WORLD
    }
}
```

---

7. remove_spaces Function

java

CopyEdit

```
public class SpaceRemover {
    public static String remove_spaces(String s) {
        return s.replace(" ", "");
    }

    public static void main(String[] args) {
        System.out.println(remove_spaces("Hello World"));       // HelloWorld
        System.out.println(remove_spaces(""));                  // (empty)
        System.out.println(remove_spaces("NoSpaces"));          // NoSpaces
        System.out.println(remove_spaces("  leading and trailing  ")); // leadingandtrailing
        System.out.println(remove_spaces("abc 123 !@#"));       // abc123!@#
    }
}
```

---

8. Iteration Examples

java

CopyEdit

```
import java.util.*;

public class IterationExamples {
    public static void main(String[] args) {
        ArrayList<String> arrayList = new ArrayList<>(List.of("A", "B", "C"));
        LinkedList<String> linkedList = new LinkedList<>(List.of("X", "Y", "Z"));
        HashMap<String, Integer> map = new HashMap<>();
        map.put("One", 1);
```

```
        map.put("Two", 2);

        // ArrayList
        for (int i = 0; i < arrayList.size(); i++) System.out.println(arrayList.get(i));
        for (String s : arrayList) System.out.println(s);
        Iterator<String> ai = arrayList.iterator();
        while (ai.hasNext()) System.out.println(ai.next());

        // LinkedList
        for (String s : linkedList) System.out.println(s);
        Iterator<String> li = linkedList.iterator();
        while (li.hasNext()) System.out.println(li.next());

        // HashMap
        for (String key : map.keySet()) System.out.println(key + ": " + map.get(key));
        for (Map.Entry<String, Integer> entry : map.entrySet())
            System.out.println(entry.getKey() + " => " + entry.getValue());
    }
}
```

---

9. Merge Two Sorted Lists

java

CopyEdit

```
import java.util.*;

public class MergeSortedLists {
    public static List<Integer> merge(List<Integer> a, List<Integer> b) {
        List<Integer> result = new ArrayList<>();
        Iterator<Integer> it1 = a.iterator();
```

```
    Iterator<Integer> it2 = b.iterator();

    Integer x = it1.hasNext() ? it1.next() : null;
    Integer y = it2.hasNext() ? it2.next() : null;

    while (x != null && y != null) {
        if (x <= y) {
            result.add(x);
            x = it1.hasNext() ? it1.next() : null;
        } else {
            result.add(y);
            y = it2.hasNext() ? it2.next() : null;
        }
    }

    while (x != null) {
        result.add(x);
        x = it1.hasNext() ? it1.next() : null;
    }

    while (y != null) {
        result.add(y);
        y = it2.hasNext() ? it2.next() : null;
    }

    return result;
}

public static void main(String[] args) {
```

```
        List<Integer> a = List.of(1, 3, 5);
        List<Integer> b = List.of(2, 4, 6);
        System.out.println(merge(a, b));  // [1, 2, 3, 4, 5, 6]
    }
}
```

---

10. Wildcards: <? extends T> and <? super T>

java

CopyEdit

```
import java.util.*;

class Animal {
    public void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    public void sound() {
        System.out.println("Bark");
    }
}

public class WildcardsExample {

    // Upper bounded wildcard
    public static void printSounds(List<? extends Animal> animals) {
        for (Animal a : animals) {
            a.sound();
```

```
        }
    }

    // Lower bounded wildcard
    public static void addDog(List<? super Dog> dogs) {
        dogs.add(new Dog());
    }

    public static void main(String[] args) {
        List<Dog> dogList = new ArrayList<>();
        addDog(dogList);  // OK
        printSounds(dogList);  // OK

        List<Animal> animals = new ArrayList<>();
        animals.add(new Dog());
        printSounds(animals);
    }
}
```

Lab 3 – Collections and Stream Exercises

Instructions

• Implement each solution in its own file.

• The use of AI software engineering tools is not allowed.

Tasks

1. Create an ArrayList that can hold objects of type String. Use the add method to add several objects to the list and iterate through the list using the foreach loop, printing the elements of the list to the console.

2. Write a program that demonstrates the creation of a LinkedList and the use of any five instance methods that come with it.

3. Create a class Student with attributes id and fullname. Create a Map that can hold objects of type Student; the key being the student ID and the value being the student object.

4. Implement a set of code that demonstrates how you can remove duplicates in a given ArrayList by creating a Set.

5. Create a class Person and add the attributes height, age, gender, and fullName. Create an ArrayList that can hold objects of type Person, add 10 objects with random values to it and then:

• Sort the list based on the ages of the people.

• Sort the list based on the heights of the people.

• Count all people who are adults.

6. Design and implement a hierarchy of geometric shape objects using object-oriented principles and implement generic methods to operate on them.

• Create an abstract base class named Shape.

• Derive two categories from it: TwoDShape and ThreeDShape.

• TwoDShape should include: Circle, SemiCircle, EquilateralTriangle, Rectangle, and Rhombus.

• ThreeDShape should include: Cube, Cone, and Cylinder.

Each shape must implement:

• A method getName() that returns the name of the shape.

• A method getArea() that calculates and returns the area (or surface area for 3D shapes).

Then implement:

• A generic method that receives a list of shapes and prints the name of each shape.

• A generic method that accepts a list of shapes, calculates the area for each, and returns the total.

• A method that receives a list of shapes and returns a map where keys are "2D" and "3D" and values are the counts of each type.

7. Design and implement a class named BrowserHistory that manages a list of visited web URLs. Implement the following methods:

• visit(url: string) – Adds the given URL to the history. Clears forward history and prints the current URL.

• back() – Navigates to the previous URL. If none, print: "No previous URL".

• forward() – Navigates forward if possible. If none, print: "No forward URL".

Hint: Use two stacks to manage back and forward navigation.

8. Create a class Patient with attributes fullName and severity. The severity is an integer (1 to 10); low values are low priority. Implement a priority queue to manage patients by severity.

Hint: Use a comparator.

9. Implement threads that can be prioritized.

10. Given a list of integers from 0 to 20, use a stream to filter and print only the numbers divisible by 2 and 5.

11. Given a list of names like "Alice", "Bob", "Andrew", "Alex", "Mark", count how many strings begin with the letter ‘A’.

12. Given a list of numbers between 100 and 1000, implement a stream that returns the first number greater than 500.

13. Implement a method that takes a Map object containing student objects (with id and

grade), calculates and returns a new map with:

- Average score ("avg")
- Median score ("median")
- Highest score ("max")
- Lowest score ("min")

Lab3 ……..

1. ArrayList with Strings

java

CopyEdit

```
import java.util.ArrayList;

public class Task1_ArrayListStrings {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

---

2. LinkedList with Five Methods

java

CopyEdit

```
import java.util.LinkedList;

public class Task2_LinkedListMethods {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Apple");
        list.addFirst("Banana");
        list.addLast("Cherry");
        list.remove("Banana");
        list.add(1, "Date");

        System.out.println("Size: " + list.size());
        for (String fruit : list) {
            System.out.println(fruit);
        }
    }
}
```

---

3. Map of Students

java

CopyEdit

```
import java.util.HashMap;
import java.util.Map;

class Student {
    int id;
    String fullname;

    Student(int id, String fullname) {
```

```
        this.id = id;
        this.fullname = fullname;
    }

    public String toString() {
        return id + ": " + fullname;
    }
}

public class Task3_StudentMap {
    public static void main(String[] args) {
        Map<Integer, Student> students = new HashMap<>();
        students.put(1, new Student(1, "Alice"));
        students.put(2, new Student(2, "Bob"));

        for (Map.Entry<Integer, Student> entry : students.entrySet()) {
            System.out.println(entry.getValue());
        }
    }
}
```

---

4. Remove Duplicates Using Set

java

CopyEdit

```
import java.util.*;

public class Task4_RemoveDuplicates {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>(List.of("apple", "banana", "apple", "orange", "banana"));
```

```
        Set<String> unique = new HashSet<>(list);

        for (String item : unique) {
            System.out.println(item);
        }
    }
}
```

---

5. Person Sorting and Counting Adults

java

CopyEdit

```
import java.util.*;

class Person {
    double height;
    int age;
    String gender;
    String fullName;

    Person(String fullName, int age, double height, String gender) {
        this.fullName = fullName;
        this.age = age;
        this.height = height;
        this.gender = gender;
    }

    public String toString() {
        return fullName + " (" + age + " yrs, " + height + "m)";
    }
```

```
}

public class Task5_PersonList {
    public static void main(String[] args) {
        ArrayList<Person> people = new ArrayList<>();
        people.add(new Person("Alice", 25, 1.65, "F"));
        people.add(new Person("Bob", 17, 1.75, "M"));
        people.add(new Person("Charlie", 30, 1.8, "M"));
        people.add(new Person("Dana", 22, 1.6, "F"));
        people.add(new Person("Eve", 15, 1.5, "F"));
        people.add(new Person("Frank", 40, 1.9, "M"));
        people.add(new Person("Grace", 28, 1.7, "F"));
        people.add(new Person("Hank", 35, 1.85, "M"));
        people.add(new Person("Ivy", 18, 1.55, "F"));
        people.add(new Person("John", 13, 1.4, "M"));

        // Sort by age
        people.sort(Comparator.comparingInt(p -> p.age));
        System.out.println("Sorted by age:");
        people.forEach(System.out::println);

        // Sort by height
        people.sort(Comparator.comparingDouble(p -> p.height));
        System.out.println("\nSorted by height:");
        people.forEach(System.out::println);

        // Count adults (18+)
        long adults = people.stream().filter(p -> p.age >= 18).count();
        System.out.println("\nNumber of adults: " + adults);
```

```
    }
}
```

---

6. Shape Hierarchy and Generics

java

CopyEdit

```
import java.util.*;

abstract class Shape {
    abstract String getName();
    abstract double getArea();
}

abstract class TwoDShape extends Shape {}
abstract class ThreeDShape extends Shape {}

class Circle extends TwoDShape {
    double radius;
    Circle(double r) { this.radius = r; }
    public String getName() { return "Circle"; }
    public double getArea() { return Math.PI * radius * radius; }
}

class Rectangle extends TwoDShape {
    double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    public String getName() { return "Rectangle"; }
    public double getArea() { return w * h; }
}
```

```
class Cube extends ThreeDShape {
    double side;
    Cube(double s) { this.side = s; }
    public String getName() { return "Cube"; }
    public double getArea() { return 6 * side * side; }
}

// More shapes can be added similarly

public class Task6_Shapes {
    public static <T extends Shape> void printNames(List<T> shapes) {
        for (T shape : shapes) {
            System.out.println(shape.getName());
        }
    }

    public static <T extends Shape> double totalArea(List<T> shapes) {
        double sum = 0;
        for (T shape : shapes) {
            sum += shape.getArea();
        }
        return sum;
    }

    public static Map<String, Long> countTypes(List<Shape> shapes) {
        long twoD = shapes.stream().filter(s -> s instanceof TwoDShape).count();
        long threeD = shapes.size() - twoD;
        Map<String, Long> map = new HashMap<>();
```

```
        map.put("2D", twoD);
        map.put("3D", threeD);
        return map;
    }

    public static void main(String[] args) {
        List<Shape> shapes = List.of(
            new Circle(3),
            new Rectangle(4, 5),
            new Cube(2)
        );

        printNames(shapes);
        System.out.println("Total area: " + totalArea(shapes));
        System.out.println("Counts: " + countTypes(shapes));
    }
}
```

---

7. Browser History with Stacks

java

CopyEdit

```
import java.util.*;

public class Task7_BrowserHistory {
    Stack<String> backStack = new Stack<>();
    Stack<String> forwardStack = new Stack<>();
    String current = null;

    public void visit(String url) {
```

```
    if (current != null) backStack.push(current);
    current = url;
    forwardStack.clear();
    System.out.println("Visiting: " + current);
}

public void back() {
    if (backStack.isEmpty()) {
        System.out.println("No previous URL");
    } else {
        forwardStack.push(current);
        current = backStack.pop();
        System.out.println("Back to: " + current);
    }
}

public void forward() {
    if (forwardStack.isEmpty()) {
        System.out.println("No forward URL");
    } else {
        backStack.push(current);
        current = forwardStack.pop();
        System.out.println("Forward to: " + current);
    }
}

public static void main(String[] args) {
    Task7_BrowserHistory history = new Task7_BrowserHistory();
    history.visit("a.com");
```

```
        history.visit("b.com");
        history.back();
        history.forward();
    }
}
```

---

8. Patient Priority Queue

java

CopyEdit

```
import java.util.*;

class Patient {
    String fullName;
    int severity;

    Patient(String fullName, int severity) {
        this.fullName = fullName;
        this.severity = severity;
    }

    public String toString() {
        return fullName + " (Severity: " + severity + ")";
    }
}

public class Task8_PatientQueue {
    public static void main(String[] args) {
        PriorityQueue<Patient> queue = new PriorityQueue<>(Comparator.comparingInt(p -> p.severity));
        queue.add(new Patient("Alice", 4));
```

```
        queue.add(new Patient("Bob", 2));
        queue.add(new Patient("Charlie", 7));

        while (!queue.isEmpty()) {
            System.out.println("Attending to: " + queue.poll());
        }
    }
}
```

---

9. Threads with Priority

java

CopyEdit

```
public class Task9_PriorityThreads {
    public static void main(String[] args) {
        Thread low = new Thread(() -> System.out.println("Low priority thread"));
        Thread high = new Thread(() -> System.out.println("High priority thread"));

        low.setPriority(Thread.MIN_PRIORITY);
        high.setPriority(Thread.MAX_PRIORITY);

        low.start();
        high.start();
    }
}
```

---

10. Stream: Filter numbers divisible by 2 and 5

java

CopyEdit

```
import java.util.*;
```

```
import java.util.stream.*;

public class Task10_StreamDivisible {
    public static void main(String[] args) {
        List<Integer> nums = IntStream.rangeClosed(0, 20).boxed().toList();
        nums.stream().filter(n -> n % 2 == 0 && n % 5 == 0).forEach(System.out::println);
    }
}
```

---

11. Count names starting with 'A'

java

CopyEdit

```
import java.util.*;

public class Task11_CountA {
    public static void main(String[] args) {
        List<String> names = List.of("Alice", "Bob", "Andrew", "Alex", "Mark");
        long count = names.stream().filter(n -> n.startsWith("A")).count();
        System.out.println("Names starting with A: " + count);
    }
}
```

---

12. First number > 500 from list

java

CopyEdit

```
import java.util.*;

public class Task12_FirstGreaterThan500 {
    public static void main(String[] args) {
```

```
        List<Integer> numbers = List.of(200, 400, 600, 800);
        numbers.stream().filter(n -> n > 500).findFirst().ifPresent(System.out::println);
    }
}
```

---

13. Student Grade Statistics

java

CopyEdit

```
import java.util.*;

class StudentGrade {
    int id;
    int grade;

    StudentGrade(int id, int grade) {
        this.id = id;
        this.grade = grade;
    }
}

public class Task13_GradeStats {
    public static Map<String, Double> calculateStats(Map<Integer, StudentGrade> studentMap) {
        List<Integer> grades = studentMap.values().stream().map(s -> s.grade).sorted().toList();
        double avg = grades.stream().mapToInt(i -> i).average().orElse(0);
        double median = grades.size() % 2 == 0 ?
            (grades.get(grades.size()/2 - 1) + grades.get(grades.size()/2)) / 2.0 :
            grades.get(grades.size()/2);
        double max = grades.get(grades.size()-1);
        double min = grades.get(0);
```

```
        Map<String, Double> stats = new HashMap<>();
        stats.put("avg", avg);
        stats.put("median", median);
        stats.put("max", max);
        stats.put("min", min);
        return stats;
    }

    public static void main(String[] args) {
        Map<Integer, StudentGrade> map = new HashMap<>();
        map.put(1, new StudentGrade(1, 90));
        map.put(2, new StudentGrade(2, 70));
        map.put(3, new StudentGrade(3, 80));

        System.out.println(calculateStats(map));
    }
}
```

Lab 2……..

Lab 2 – Implementing Generic Classes and Methods

1. Implement a generic class called Generic that can hold any type of object.

▪ Create a class Generic with a type parameter T.

▪ Add methods to set and get the value of the type parameter.

▪ Write a Main class to test the creation of instances of Generic with different types.

2. Implement a generic class named RectangleDimension that holds two instance variables: length and width. Since dimensions are numeric, restrict the class to only

accept types that extend the Number class.

3. Create a generic class Circle that stores a radius. Restrict the class to accept only types that extend Number. Add a method to compute and return the area of the circle.

4. Write a generic method sumList that takes a List<? extends Number> and returns the total sum as a double.

5. Implement a class named StudentList that maintains a list of Student objects. The list should support storing instances of subclasses such as UndergraduateStudent and PostGraduateStudent. Include methods to:

- Add a Student object to the list
- Remove a Student object from the list
- Optionally, implement a method to list all students or filter them by type

6. Implement methods generic methods with the following specifications.

a. A method called reverseList(…) that receives a list of items of any types and returns a list of the items in reverse order.

b. A method called countOccurences(…) that receives a list of items of type T and an element of type T. The method counts the number of times the element is in the list and returns the count.

c. A method called mergeLists(…) that receives two list containing type T objects and returns a merged list containing the elements in the given lists.

7. Create a bounded type calculator called Calculator<…> that works with two types that extends Number. Implement the following methods.

8. Implement a generic stack data structure Stack<T> that contains a list keeping elements of type T. add the following stack related methods. Hint: use the array list.

- push(T item), receives an element of type T and adds it to the top of the stack.
- pop(), removes the first element in the stack.
- peek(), prints the first element in the stack.
- isEmpty(), returns true if the stack is empty else it returns false if the stack is not empty.

1. Generic Class

java

CopyEdit

```
// Generic.java
public class Generic<T> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Generic<String> g1 = new Generic<>();
        g1.setValue("Hello");
        System.out.println(g1.getValue());

        Generic<Integer> g2 = new Generic<>();
        g2.setValue(100);
        System.out.println(g2.getValue());
    }
}
```

---

2. RectangleDimension Class

java

CopyEdit

```
public class RectangleDimension<T extends Number> {
    private T length;
    private T width;

    public RectangleDimension(T length, T width) {
        this.length = length;
        this.width = width;
    }

    public T getLength() {
        return length;
    }

    public T getWidth() {
        return width;
    }

    public double getArea() {
        return length.doubleValue() * width.doubleValue();
    }
}
```

---

3. Circle Class

java

CopyEdit

```
public class Circle<T extends Number> {
    private T radius;

    public Circle(T radius) {
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * Math.pow(radius.doubleValue(), 2);
    }

    public T getRadius() {
        return radius;
    }
}
```

---

4. sumList Method

java

CopyEdit

```
import java.util.List;

public class NumberUtils {
    public static double sumList(List<? extends Number> numbers) {
        double sum = 0;
        for (Number num : numbers) {
            sum += num.doubleValue();
        }
        return sum;
    }
```

}

---

5. StudentList with Subclasses

java

CopyEdit

```
import java.util.ArrayList;
import java.util.List;

class Student {
    String name;
    Student(String name) { this.name = name; }

    public String toString() {
        return "Student: " + name;
    }
}

class UndergraduateStudent extends Student {
    UndergraduateStudent(String name) { super(name); }
}

class PostGraduateStudent extends Student {
    PostGraduateStudent(String name) { super(name); }
}

public class StudentList {
    private List<Student> students = new ArrayList<>();

    public void add(Student student) {
```

```
        students.add(student);
    }

    public void remove(Student student) {
        students.remove(student);
    }

    public void listAll() {
        for (Student s : students) {
            System.out.println(s);
        }
    }

    public void listByType(Class<?> type) {
        for (Student s : students) {
            if (type.isInstance(s)) {
                System.out.println(s);
            }
        }
    }
}
```

---

6. Generic Utility Methods

java

CopyEdit

```
import java.util.*;

public class GenericUtils {
    // a. reverseList
```

```
    public static <T> List<T> reverseList(List<T> list) {
        List<T> reversed = new ArrayList<>(list);
        Collections.reverse(reversed);
        return reversed;
    }

    // b. countOccurrences
    public static <T> int countOccurrences(List<T> list, T element) {
        int count = 0;
        for (T item : list) {
            if (item.equals(element)) {
                count++;
            }
        }
        return count;
    }

    // c. mergeLists
    public static <T> List<T> mergeLists(List<T> l1, List<T> l2) {
        List<T> merged = new ArrayList<>(l1);
        merged.addAll(l2);
        return merged;
    }
}
```

---

7. Bounded Calculator

java

CopyEdit

```
public class Calculator<T extends Number, U extends Number> {
```

```
    private T value1;
    private U value2;

    public Calculator(T value1, U value2) {
        this.value1 = value1;
        this.value2 = value2;
    }

    public double add() {
        return value1.doubleValue() + value2.doubleValue();
    }

    public double subtract() {
        return value1.doubleValue() - value2.doubleValue();
    }

    public double multiply() {
        return value1.doubleValue() * value2.doubleValue();
    }

    public double divide() {
        if (value2.doubleValue() == 0) throw new ArithmeticException("Divide by zero");
        return value1.doubleValue() / value2.doubleValue();
    }
}
```

---

8. Generic Stack

java

CopyEdit

```java
import java.util.ArrayList;

public class Stack<T> {
    private ArrayList<T> items = new ArrayList<>();

    public void push(T item) {
        items.add(item);
    }

    public T pop() {
        if (!items.isEmpty()) {
            return items.remove(items.size() - 1);
        }
        return null;
    }

    public T peek() {
        if (!items.isEmpty()) {
            return items.get(items.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return items.isEmpty();
    }
}
```

Lab 1 – Implementing methods and unit tests

1. Write a method that takes an array of numbers, calculates the sum of the numbers and returns the sum. Consider the following test cases.

- An array of size 4 with positive integers
- An array of any size with mixed numbers
- An of any size with zeros
- Any array full of large numbers
- An empty array

Note: The method should be throwing exceptions in any odd circumstances.

2. Implement a method that finds the maximum value in an array (or list) of numbers. The following are the requirements for the method.

- It should return the largest number in the array.
- If the array is empty, it should throw an Exception.
- Handle arrays with positive, negative, and mixed numbers.

Implement the following test cases.

- Test with [1, 5, 2, 8, 3]
- Test with [-10, -5, -20]
- Test with [0, 0, 0].
- Test with [7].
- Test with an empty array [] (expecting an error).
- Test with an array containing big numbers.

3. Implement methods, design test cases and implement unit tests for each of the following method specifications.

- A method that receives are array and a search item, returns true if an item exist in the array or false if it does not.
- A method that receives an array and returns an array that is the reverse of the received array.
- A method that receives a birth year, calculates the age of a person and returns it.

4. Create a class called DiscountCalculator that contains a method float

calculateFinalPrice(float price, String customerType). This method should take the original price of a product and the type of customer as inputs and return the final price after applying the appropriate discounts. The following are the customer types and their discounts.

Customer type Discount

Regular 5%

Member 10%

VIP 20%

Employee 30%

New 0%

Task: Design test cases and implement them. The tests should result into 100% code coverage. In your design consider the following conditions.

• It should be case sensitive to customer type value (“Regular” is not equal to “regular”)

• For invalid customer type return the original price

• For invalid price return 0

5. Write a method remove_spaces(String s) that returns the string s with all spaces removed (do not use an inbuilt function).

Test cases:

a) Test removing spaces from a normal string.

b) Test removing spaces from an empty string.

c) Test removing spaces from a string with no spaces.

d) Test removing spaces from a string with leading and trailing spaces.

e) Test removing spaces from a string with special characters and numbers.

6. Write a method that calculates a student's final grade based on three scores: midterm, project, and final exam. The midterm counts for 30%, the project for 20%, and the final exam for 50%. Use the following method signature:

Consider the following test cases.

▪ Test a midterm score of 80, project score of 90, and final exam score of 70.

- Test a midterm score of 50, project score of 60, and final exam score of 40.
- Test a midterm score of 0, project score of 0, and final exam score of 0.
- Test a perfect score on all three (100, 100, 100).
- Test with negative scores to validate input checking.

7. Regarding the implementation of tests in task 4, are you confident that test you implemented covered enough scenarios to consider the method ready for use? If not what recommend the tests that need to be done.

8. Create a method that calculates the monthly payment for a loan given the principal amount, annual interest rate, and number of months to pay off the loan. Use the following method signature:

Where:

- M is the monthly payment
- P is the principal loan amount
- r is the monthly interest rate (annual interest rate divided by 12)
- n is the total number of payments (number of months)

9. Implement a method to validate if an email address has a basic valid format. This is a simplified validation, so don't worry about every test case, but cover common patterns. Consider the following requirements.

- An email must contain exactly one @ symbol.
- An email must contain at least one . after the @ symbol.
- An email cannot start or end with @ or ..
- An email should not contain spaces.
- If email is null, undefined, or not a string, it should throw an Error.

Implement a method called isValidEmail(String email) which passes the following unit tests.

- Valid emails like test@example.com, user.name@sub.domain.co.uk
- Missing the @ like testexample.com
- Missing a . like test@examplecom
- Emails with multiple @ like test@example@com

▪ Emails starting with @ or . like @example.com, .example.com

▪ Emails ending with @ or . like test@example., test@example.com@

▪ Test with a null value

▪ Test with an empty string

10. Implement a function that calculates the number of working days (Monday-Friday) between two given dates (inclusive). Consider the following requirements.

▪ Both startDate and endDate are inclusive.

▪ The function should return 0 if startDate is after endDate.

▪ Input startDate and endDate must be valid Date objects (or equivalent). If not, throw an Exception.

Implement the following unit tests.

▪ Test a short period within the same week (e.g., Monday to Friday).

▪ Test a period spanning weekends (e.g., Friday to Monday).

▪ Test a period across multiple weeks.

▪ Test a startDate that is a weekend day.

▪ Test an endDate that is a weekend day.

▪ Test startDate and endDate being the same day (and it's a weekday vs. weekend).

▪ Test startDate is after endDate (expecting 0).

▪ Test with invalid date inputs like nulls

Lab1…….

1. Method to calculate the sum of an array

java

CopyEdit

```
public class ArrayUtils {

    public static int calculateSum(int[] numbers) {
        if (numbers == null) {
```

```
        throw new IllegalArgumentException("Array cannot be null");
    }
    int sum = 0;
    for (int num : numbers) {
        sum += num;
    }
    return sum;
  }
}
```

Test cases:

java

CopyEdit

```
// JUnit 5
@Test
void testCalculateSumPositive() {
  assertEquals(10, ArrayUtils.calculateSum(new int[]{1, 2, 3, 4}));
}

@Test
void testCalculateSumMixed() {
  assertEquals(2, ArrayUtils.calculateSum(new int[]{3, -1, 0}));
}

@Test
void testCalculateSumZeros() {
  assertEquals(0, ArrayUtils.calculateSum(new int[]{0, 0, 0}));
}

@Test
```

```
void testCalculateSumEmpty() {
    assertEquals(0, ArrayUtils.calculateSum(new int[]{}));
}
```

✅ **2. Method to find maximum in array**

java

CopyEdit

```
public static int findMax(int[] numbers) {
    if (numbers == null || numbers.length == 0) {
        throw new IllegalArgumentException("Array cannot be empty or null");
    }
    int max = numbers[0];
    for (int num : numbers) {
        if (num > max) {
            max = num;
        }
    }
    return max;
}
```

Test cases:

java

CopyEdit

```
@Test
void testMaxPositive() {
    assertEquals(8, findMax(new int[]{1, 5, 2, 8, 3}));
}

@Test
void testMaxNegative() {
```

```
    assertEquals(-5, findMax(new int[]{-10, -5, -20}));
}

@Test
void testMaxZeros() {
    assertEquals(0, findMax(new int[]{0, 0, 0}));
}

@Test
void testMaxSingle() {
    assertEquals(7, findMax(new int[]{7}));
}

@Test
void testMaxEmpty() {
    assertThrows(IllegalArgumentException.class, () -> findMax(new int[]{}));
}
```

✅ **3. Utility Methods**

a) Search in Array:

java

CopyEdit

```
public static boolean contains(int[] arr, int target) {
    for (int num : arr) {
        if (num == target) return true;
    }
    return false;
}
```

b) Reverse Array:

java

CopyEdit

```
public static int[] reverse(int[] arr) {
    int[] result = new int[arr.length];
    for (int i = 0; i < arr.length; i++) {
        result[i] = arr[arr.length - 1 - i];
    }
    return result;
}
```

c) Calculate Age:

java

CopyEdit

```
public static int calculateAge(int birthYear) {
    int currentYear = java.time.Year.now().getValue();
    if (birthYear > currentYear) throw new IllegalArgumentException("Birth year cannot be in the future");
    return currentYear - birthYear;
}
```

✅ **4. DiscountCalculator**

java

CopyEdit

```
public class DiscountCalculator {

    public float calculateFinalPrice(float price, String customerType) {
        if (price < 0) return 0;

        switch (customerType) {
            case "Regular": return price * 0.95f;
            case "Member": return price * 0.90f;
```

```
            case "VIP": return price * 0.80f;
            case "Employee": return price * 0.70f;
            case "New": return price;
            default: return price;
        }
    }
}
```

Tests (sample):

java

CopyEdit

```
@Test
void testVIP() {
    assertEquals(80.0f, new DiscountCalculator().calculateFinalPrice(100, "VIP"));
}

@Test
void testInvalidCustomerType() {
    assertEquals(100.0f, new DiscountCalculator().calculateFinalPrice(100, "guest"));
}

@Test
void testInvalidPrice() {
    assertEquals(0, new DiscountCalculator().calculateFinalPrice(-5, "Regular"));
}
```

✅ 5. Remove Spaces

java

CopyEdit

```
public static String removeSpaces(String s) {
```

```
    if (s == null) return null;
    StringBuilder result = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (c != ' ') result.append(c);
    }
    return result.toString();
}
```

✅ 6. Final Grade Calculator

java

CopyEdit

```
public static double calculateFinalGrade(double midterm, double project, double finalExam) {
    if (midterm < 0 || project < 0 || finalExam < 0) throw new IllegalArgumentException("Negative scores are not allowed");
    return midterm * 0.3 + project * 0.2 + finalExam * 0.5;
}
```

✅ 7. Test Coverage Analysis for DiscountCalculator

Yes, if you've tested:

- All valid customer types (5 tests)
- Invalid type (1 test)
- Negative price (1 test)

You have **100% branch and condition coverage**.

✅ 8. Monthly Loan Payment

java

CopyEdit

```
public static double calculateMonthlyPayment(double principal, double annualRate, int months) {
    if (principal < 0 || annualRate < 0 || months <= 0)
```

```
        throw new IllegalArgumentException("Invalid input");

    double monthlyRate = annualRate / 12 / 100;
    return (principal * monthlyRate) / (1 - Math.pow(1 + monthlyRate, -months));
}
```

✅ **9. Email Validator**

java

CopyEdit

```
public static boolean isValidEmail(String email) {
    if (email == null || email.trim().isEmpty()) throw new IllegalArgumentException("Invalid email");

    if (email.contains(" ") || email.startsWith("@") || email.startsWith(".")
            || email.endsWith("@") || email.endsWith(".")) return false;

    int atCount = email.length() - email.replace("@", "").length();
    if (atCount != 1) return false;

    int indexAt = email.indexOf("@");
    return email.indexOf(".", indexAt) != -1;
}
```

✅ **10. Working Days Between Two Dates**

java

CopyEdit

```
public static long countWorkingDays(LocalDate start, LocalDate end) {
    if (start == null || end == null)
        throw new IllegalArgumentException("Date cannot be null");
```

```
    if (start.isAfter(end)) return 0;

    long daysBetween = ChronoUnit.DAYS.between(start, end) + 1;

    long workingDays = 0;
    for (int i = 0; i < daysBetween; i++) {
        DayOfWeek day = start.plusDays(i).getDayOfWeek();
        if (day != DayOfWeek.SATURDAY && day != DayOfWeek.SUNDAY) {
            workingDays++;
        }
    }
    return workingDays;
}
```